

LAB 10

NAME: AADITYA AMLAN PANDA

ROLL: 220007

INTRODUCTION

To implement a classifier using different types of loss functions and to note their parameters

METHODOLOGY

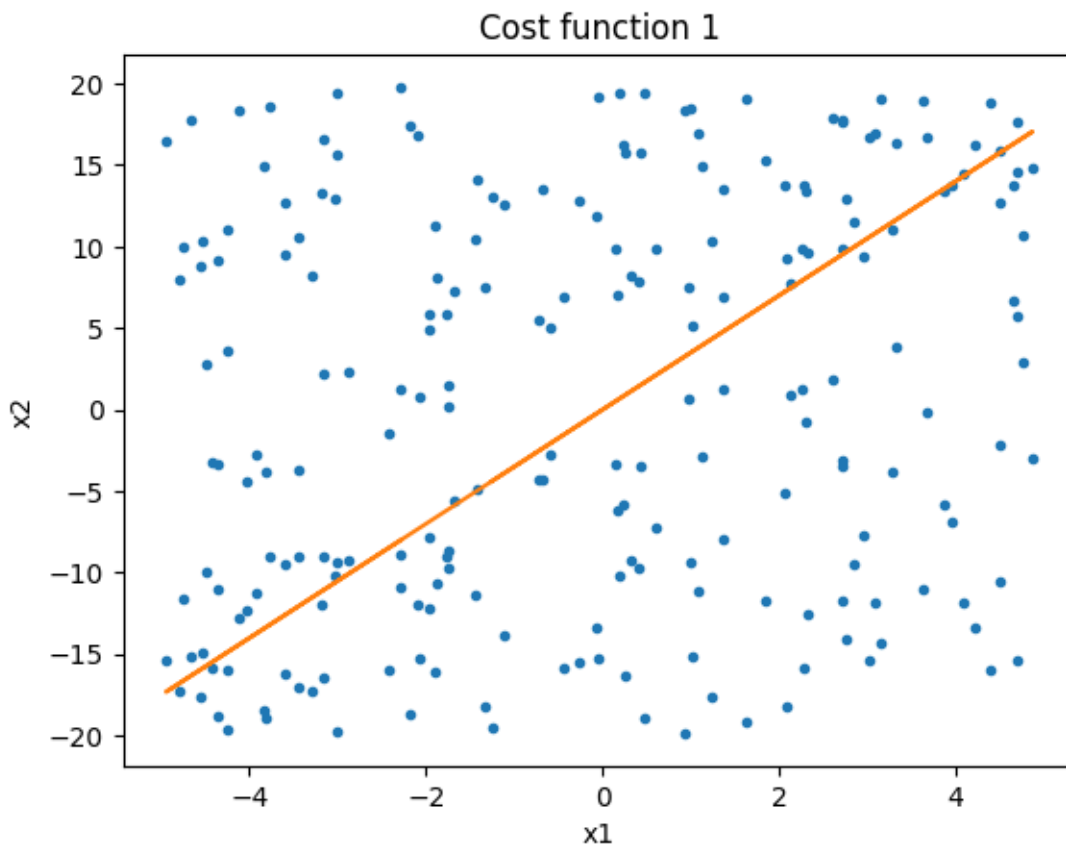
1. Our goal is to find $X'w = 0$, a linear decision boundary in case of 4 different types of cost functions.
2. Gradient Descent is performed on each of the function to estimate the parameters w and b (also embedded inside w)
3. The decision boundary is plotted along with the data points

RESULTS

Using, initial guess of $w = [1,1]$ (or $w = [1,1,1]$, when b was included), learning rate of 0.001, error threshold of 0.001 and lambda value of 0.001(in function 2 and 4: The last value in w corresponds to b) :-

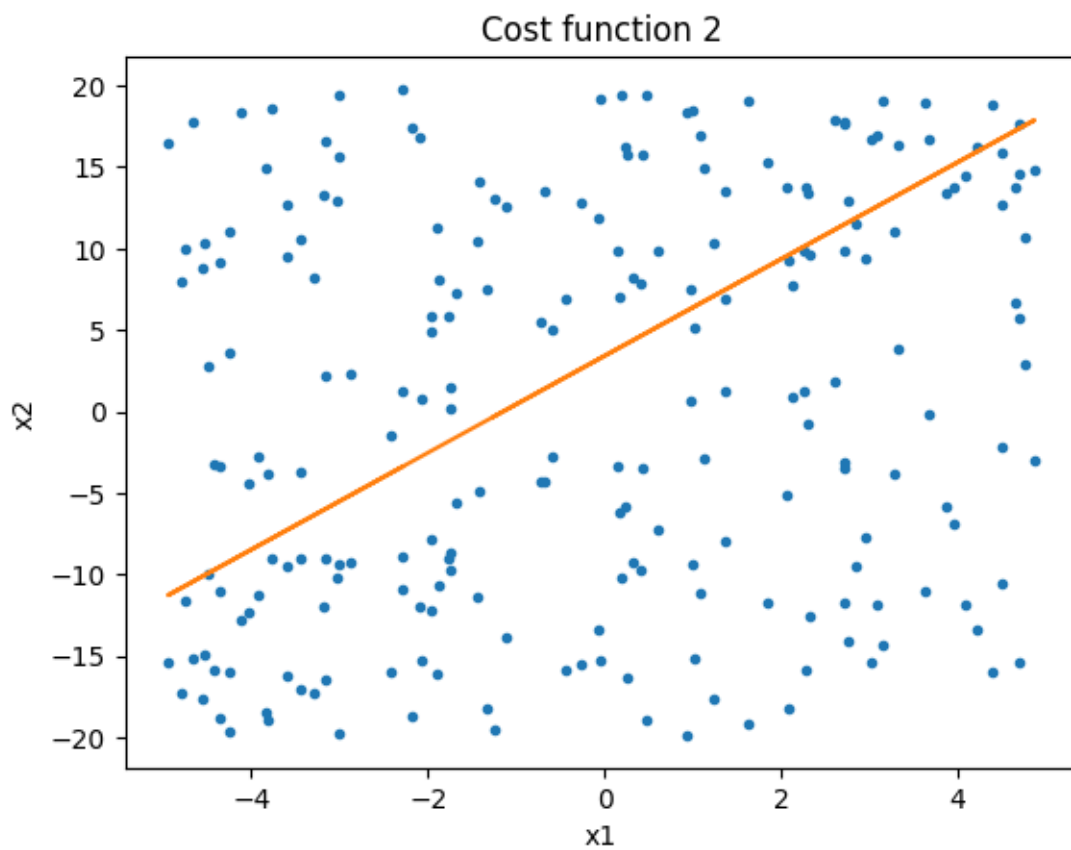
For Cost Function 1 :-

$w = [0.848 \ -0.242]$



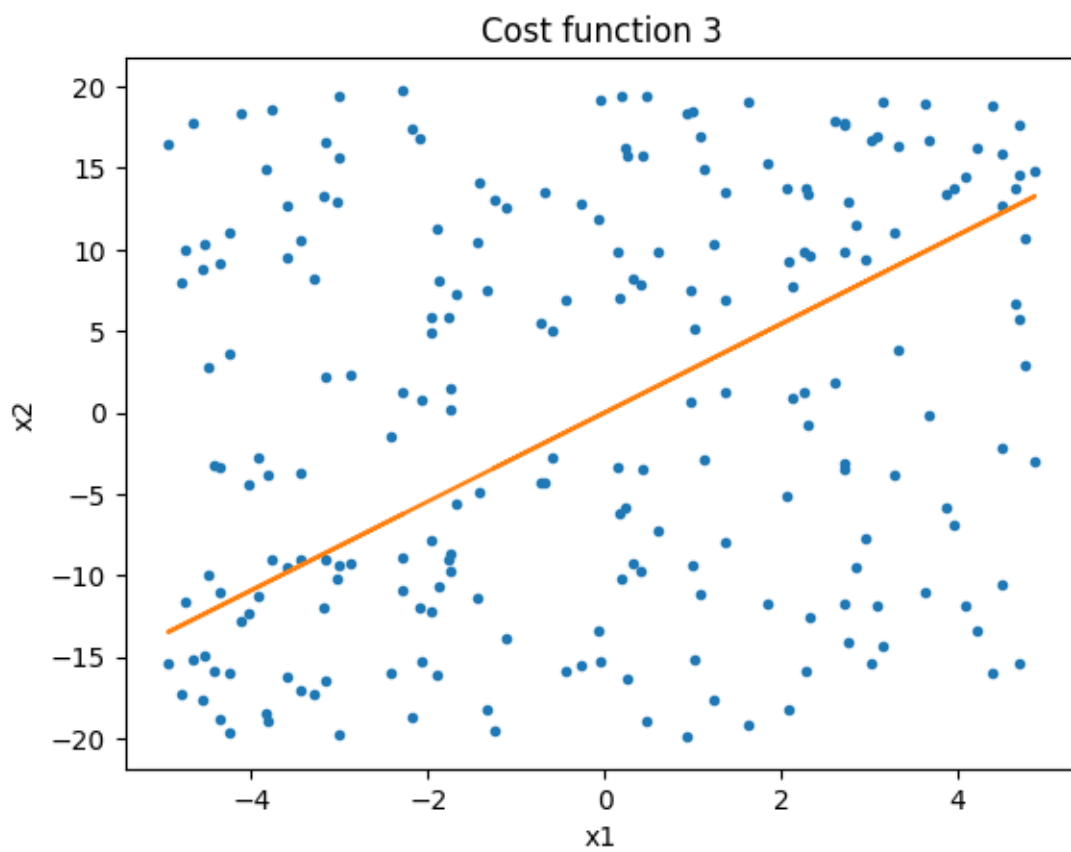
For Cost Function 2 :-

$w = [0.857 \ -0.289 \ 0.987]$



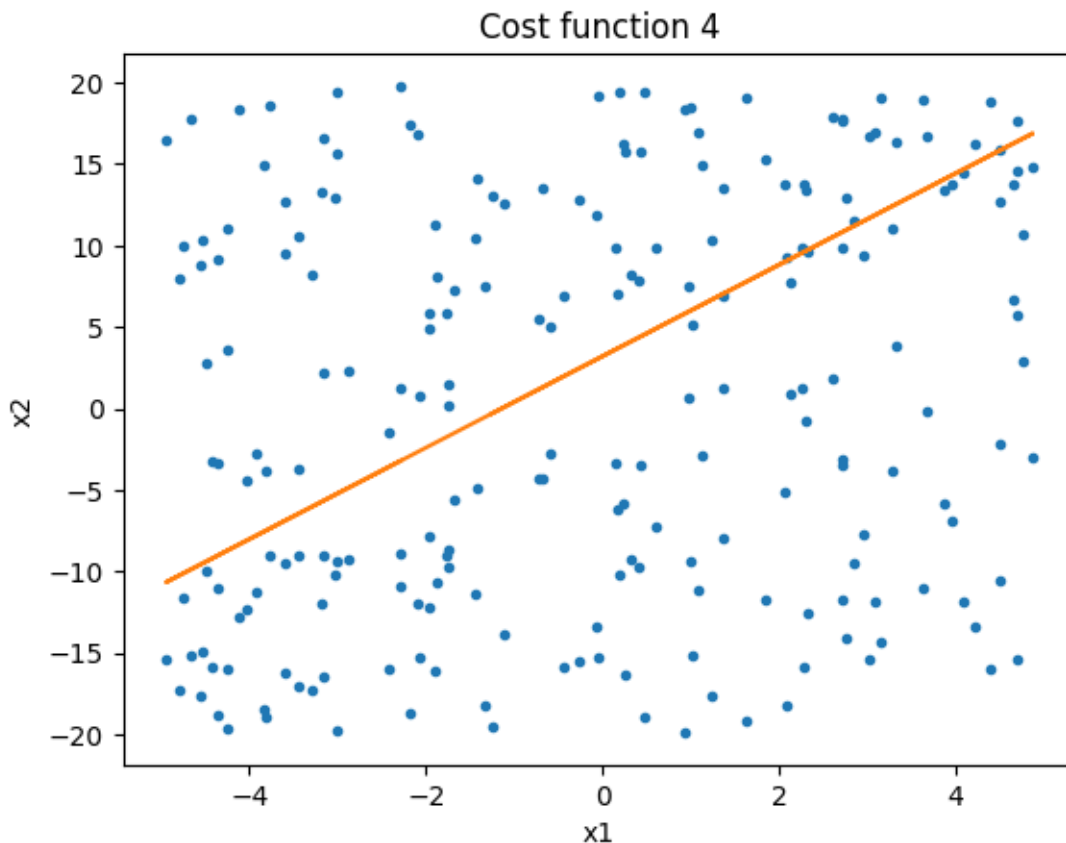
For Cost Function 3 :-

$w = [0.849 \ -0.312]$



For Cost Function 4 :-

$w = [0.861 \ -0.307 \ 0.987]$



OBSERVATIONS

- The solutions for 1 and 2 were not same but very close up to two significant figures. b was unique to function 2 as defined by the problem itself.
- The solutions were a bit different in 3 and 4 in case of w_2 but their w were quite close. B in cost function 2 and 4 were equal up to 3 significant figures.
- Minima depended a lot on initial guess, error threshold, and learning rate.
- Solutions could be obtained for each of the four cases

APPENDIX

```
import numpy as np
np.set_printoptions(precision = 3,
suppress = True)
import pandas as pd
import matplotlib.pyplot as mp

def grad_cf1(y,X,w):
    n = y.shape[0]
    grad = np.zeros(2)
    for p in range(n):
        check = y[p]*(X[p,0]*w[0]+X[p,1]*w[1])
        if check < 0:
            grad[0]-=y[p]*X[p,0]
            grad[1]-=y[p]*X[p,1]
```

```
    return grad/n
```

```
def grad_cf2(y,X,w,lda):
    n = y.shape[0]
    grad = np.zeros(3)
    grad[0] = 2*lda*w[0]
    grad[1] = 2*lda*w[1]
    for p in range(n):
        check = y[p]*(X[p,0]*w[0]+X[p,1]*w[1])
        sigma = np.exp(-y[p]*w[2] - check)
        grad[0]-=y[p]*X[p,0]*sigma/(1+sigma)/n
        grad[1]-=y[p]*X[p,1]*sigma/(1+sigma)/n
        grad[2]-=y[p]*sigma/(1+sigma)/n
```

```

return grad

def grad_cf3(y,X,w):
    n = y.shape[0]
    grad = np.zeros(2)
    for p in range(n):
        check = y[p]*(X[p,0]*w[0]+X[p,1]*w[1])
        if check <1:
            grad[0]-=y[p]*X[p,0]
            grad[1]-=y[p]*X[p,1]
    return grad/n

def grad_cf4(y,X,w,lda):
    n = y.shape[0]
    grad = np.zeros(3)
    grad[0] = 2*lda*w[0]
    grad[1] = 2*lda*w[1]
    for p in range(n):
        check_new = 1-y[p]*(w[2] +
        X[p,0]*w[0]+X[p,1]*w[1])
        if check_new >0:
            grad[0]-=y[p]*X[p,0]/n
            grad[1]-=y[p]*X[p,1]/n
            grad[2]-=y[p]/n
    return grad

dat =
pd.read_csv('/content/dataset_lab10.csv').values
x = dat[:,0:2]
y = dat[:,2]
N = len(x)
s = 1e-3 #learning rate
eps = 1e-3 #error threshold
lda = 1e-3

# Cost function 1
w = np.ones(2)
wold = np.zeros(2)
iter = 0

while np.linalg.norm(w-wold) > eps:
    iter+=1
    wold = w
    grad = grad_cf1(y,x,w)
    w = wold-s*grad
    print(w)

    mp.plot(x[:,0],x[:,1], '.')
    mp.plot(x[:,0],-w[0]*x[:,0]/w[1])
    mp.xlabel('x1')

mp.ylabel('x2')
mp.title('Cost function 1')
mp.show()

# Cost function 2
w = np.ones(3)
wold = np.zeros(3)

while np.linalg.norm(w-wold) > eps:
    wold = w
    grad = grad_cf2(y,x,w,lda)
    w = wold-s*grad
    print(w)

    mp.plot(x[:,0],x[:,1], '.')
    mp.plot(x[:,0],-(w[0]*x[:,0]+w[2])/w[1])
    mp.xlabel('x1')
    mp.ylabel('x2')
    mp.title('Cost function 2')
    mp.show()

# Cost function 3
w = np.ones(2)
wold = np.zeros(2)
iter = 0

while np.linalg.norm(w-wold) > eps:
    iter += 1
    wold = w
    grad = grad_cf3(y,x,w)
    w = wold-s*grad
    print(w)

    mp.plot(x[:,0],x[:,1], '.')
    mp.plot(x[:,0],-w[0]*x[:,0]/w[1])
    mp.xlabel('x1')
    mp.ylabel('x2')
    mp.title('Cost function 3')
    mp.show()

# Cost function 4
w = np.ones(3)
wold = np.zeros(3)
iter = 0

while np.linalg.norm(w-wold) > eps:
    iter += 1
    wold = w
    grad = grad_cf4(y,x,w,lda)
    w = wold-s*grad

```

```
print(w)

mp.plot(x[:,0],x[:,1], '.')
mp.plot(x[:,0], -(w[0]*x[:,0]+w[2])/
w[1])
mp.xlabel('x1')

mp.ylabel('x2')
mp.title('Cost function 4')
mp.show()
```